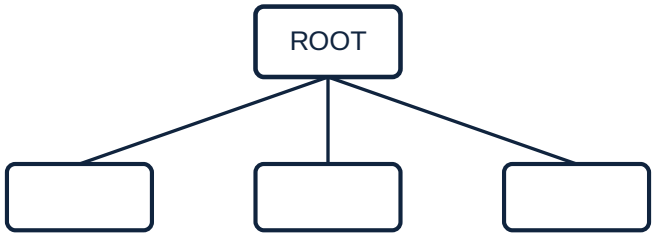
DATABASE MANAGEMENT SYSTEMS — QUICK STUDY NOTES

Chapter 3

Hierarchical Database

3.1 Introduction — Short Notes



Condensed Revision Edition for

OAVS · KVS · NVS · DSSSB · OSSSC · B.Ed Entrance · MCA · BCA · B.Tech

Chapter 3: Hierarchical Database

Short, exam-focused notes covering only the essential concepts of 3.1 Introduction.

3.1 Introduction

Definition

Definition

A **Hierarchical Database** organizes data in a **tree-like structure**, where every child record has exactly **one parent** record, forming a **one-to-many (1:N)** relationship. The topmost record is called the **root node**.

Introduction & Background

The hierarchical model is one of the earliest database models, introduced in the late 1960s and popularized by **IBM's IMS (Information Management System)**, built for the Apollo space program. Data is arranged in levels — similar to a family tree or a computer's folder structure — where each parent can have many children, but every child belongs to exactly one parent. It was later largely replaced by the more flexible **relational model**, but it still forms the conceptual base for tree-structured data such as org charts and file systems.

Exam Point

Hierarchical Model = **Tree structure** + **Parent–Child Relationship (1:N)**. This single line answers most definition-based questions.

Structure — Root, Parent, Child & Levels

- **Root Node:** The topmost node; has no parent (Level 0).
- **Parent Node:** A node with one or more child nodes linked below it.
- **Child Node:** A node linked below a parent; every child has exactly one parent.
- **Leaf Node:** A node with no children (bottom of a branch).
- **Level:** A node's depth from the root (root = Level 0, its children = Level 1, and so on).

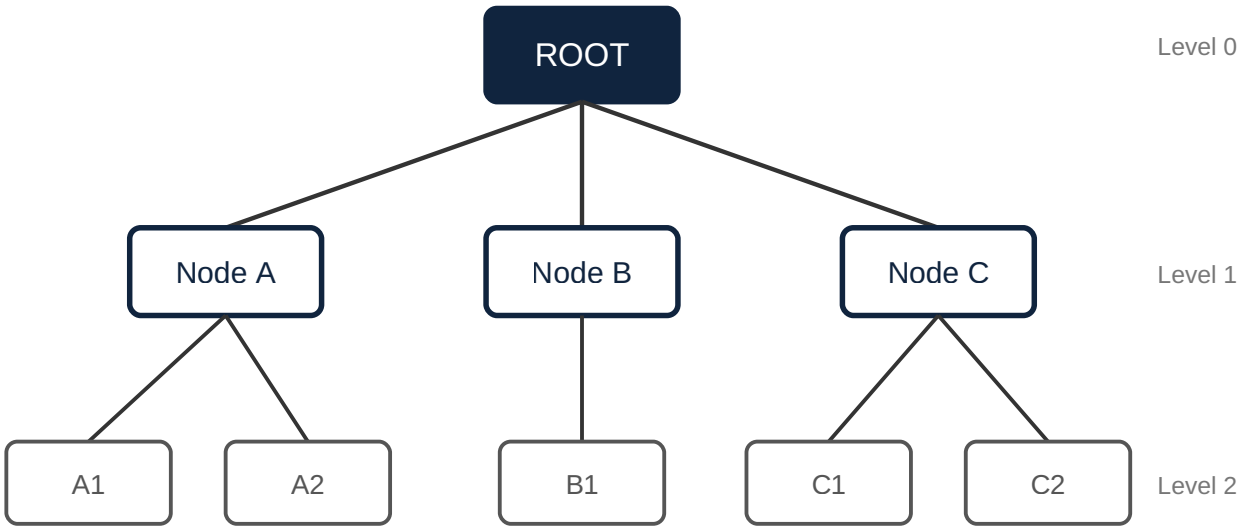


Fig. 3.1 — Basic Tree Structure of a Hierarchical Database

The Root Node lies at Level 0. Nodes A, B and C at Level 1 are children of the Root. Nodes A1, A2, B1, C1 and C2 at Level 2 are children of their respective Level-1 parents. Every child node connects to exactly one parent, forming an inverted-tree shape.

Parent–Child Relationship (1:N)

Every parent may have **many** children, but every child has **only one** parent. This single rule defines the entire hierarchical model and governs how data is accessed — always starting at the root and moving downward, level by level.

> ⚠ **Common Mistake**
>
> A child having more than one parent is **not allowed** in the pure hierarchical model — that belongs to the Network Model, which is a different topic.

Tree Structure & One-to-Many Relationship

The root, parent, and child nodes together form a complete **tree structure** — an "inverted tree" with the root at the top and branches spreading downward. This entire structure is built on the **one-to-many (1:N)** relationship: one parent may connect to many children, but the reverse is never true.

Need for the Hierarchical Model

It was developed to replace slow, redundant **flat-file** storage with a structure that naturally represents real-world one-to-many data (e.g., org charts, file systems), giving faster retrieval and better-organized, less redundant data.

Data Access Method

Access always begins at the **root node** and moves downward, level by level (top-to-bottom, left-to-right). A child record can **never** be reached directly — the system must first pass through its parent.

Characteristics & Features (Summary)

Point	Explanation
Tree-based structure	Inverted tree with a single root node.
One parent per child	Strict 1:N parent–child linkage.
Fixed access path	Data must be accessed from the root downward.
Rigid schema	Tree shape must be defined in advance; hard to change later.
Pointer/link-based navigation	Uses physical or logical pointers to connect parent and child records.
Segment-based design	Record types organized as hierarchical "segments" (IBM IMS terminology).

Advantages

- Simple, intuitive tree structure, easy to visualize.
- Fast retrieval since access paths are predefined.
- Naturally efficient for one-to-many relationships.
- Strong data integrity — every child is tied to a specific parent.
- **Security:** access can be restricted to specific segments of the tree, making hierarchy-based access control easier to enforce.
- **Performance:** delivers high throughput for fixed-relationship, high-volume transaction systems such as banking and airline reservations.

Disadvantages

- Rigid and inflexible; difficult to restructure once designed.
- Cannot directly represent many-to-many relationships.
- Possible data redundancy if data logically belongs under multiple parents.
- Searching outside the parent-to-child path is inefficient.
- **Query limitations:** requires a specific, less flexible query language compared to SQL used in relational databases, restricting the depth of data analysis.

Applications & Real-life Examples

Example	Hierarchy
Company organization	CEO → Manager → Employee
File system	Drive → Folder → File
Family tree	Grandparent → Parent → Child
Banking system	Head Office → Zonal Office → Branch → Account
Content Management Systems (CMS)	Site → Category → Page/Article → Media
Geographic Information Systems (GIS)	Map → Layer (terrain/roads) → Landmark
Network & telecom management	Network → Segment → Switch/Router

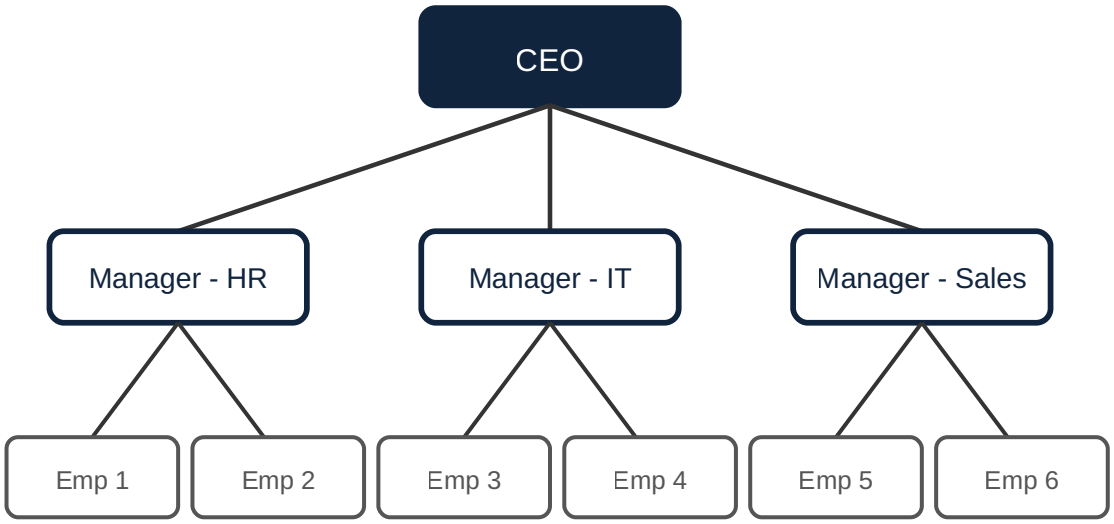


Fig. 3.2 — Company Employee Hierarchy (Organization Chart Example)

The CEO is the root. Each Manager is a child of the CEO and, in turn, becomes a parent for the Employees reporting to that department — a classic multi-level 1:N hierarchy.

Important Points for Exams

Must-Remember Facts

- A child can have **only one parent** — the most frequently tested fact.
- Root node has **no parent** and lies at Level 0.
- **IBM IMS** is the classic real-world hierarchical DBMS.
- Access always proceeds **from the root downward**.
- Replaced mostly by the **relational model** (Codd, 1970) due to rigidity.

Quick Revision — One-Line Points

1. Hierarchical database = tree structure with a single root node.
2. Every child has exactly one parent (strict 1:N rule).
3. One parent can have many children.
4. Root node = Level 0, no parent.
5. Leaf node = a node with no children.
6. IBM IMS (c. 1966) popularized this model, built for the Apollo program.
7. Data access always starts at the root and moves downward.
8. Cannot represent many-to-many relationships directly.
9. Mostly replaced today by the relational model.
10. Real examples: org charts, file systems, family trees, banking hierarchy.

Final Reminder

Everything in this chapter comes back to one rule: **"One Parent, Many Children — Every Child Has Exactly One Parent."**

Still Relevant Today

Although mostly replaced by the relational model, the hierarchical approach still underlies systems that need rigid, predictable structure — such as XML data processing, Content Management Systems, and file systems.

Reference: GeeksforGeeks, *"Hierarchical Model in DBMS"* — www.geeksforgeeks.org/dbms/hierarchical-model-in-dbms/